Bounded Types and Wildcards (PECS)

10 min read · Updated 28 Jul 2026 · Topic 24 of 65

Why it matters

Wildcards are where generics stop being obvious. `? extends` and `? super` feel arbitrary until you see the rule behind them, and then they become mechanical. Interviewers ask because PECS is a genuine "do you understand the type system or did you copy a signature from Stack Overflow?" test — and because you cannot read the JDK's own APIs (`Collections.copy`, `Stream.map`, `Comparator.comparing`) without it.

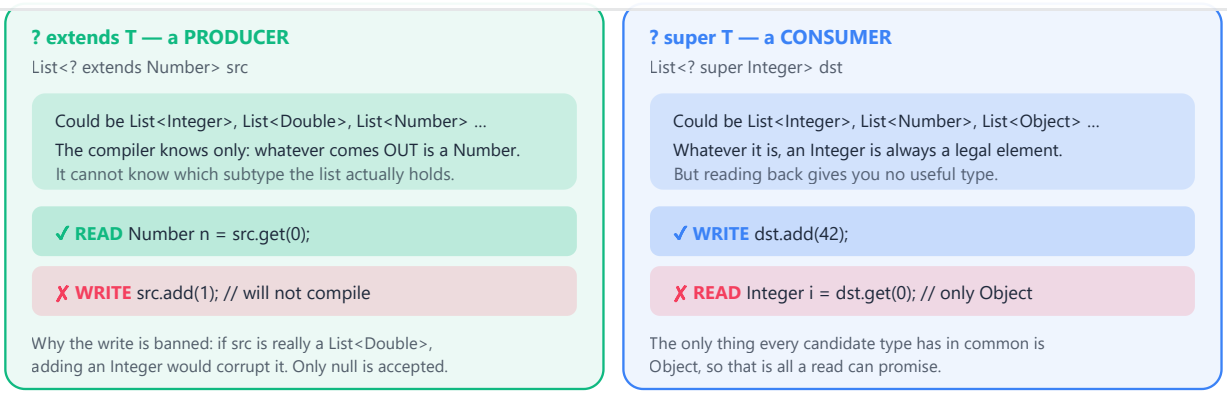
The concept, simply

Generics are **invariant**: `List<String>` is not a `List<Object>`, even though `String` is an `Object`. That is deliberate — if it were allowed, you could add an `Integer` through the `List<Object>` reference and corrupt the list.

Wildcards restore the flexibility without the hole, in one of two directions:

- `? extends T` — "some unknown subtype of T". You can **read** T out of it. You cannot write into it.
- `? super T` — "some unknown supertype of T". You can **write** T into it. Reading gives only `Object`.

The mnemonic is **PECS — Producer Extends, Consumer Super**. If the parameter *produces* values for you to consume, use `extends`. If it *consumes* values you supply, use `super`.



Both restrictions come from the same reasoning: the compiler knows only one end of the type range, so it permits exactly the operation that is safe for every type the wildcard could stand for.

How it works

Work through *why* each restriction exists and you will never need the mnemonic again.

Given `List<? extends Number> src`, the actual object might be a `List<Integer>`, a `List<Double>`, or a `List<Number>`. **Reading** is safe: whichever it is, every element is a `Number`. **Writing** is not: if you add an `Integer` and the list is really a `List<Double>`, you have corrupted it. The compiler cannot tell which, so it forbids all writes except `null`.

Given `List<? super Integer> dst`, the object might be a `List<Integer>`, `List<Number>`, or `List<Object>`. **Writing** an `Integer` is safe in all three. **Reading** is not useful: the only type all three share is `Object`.

Bounded type parameters are the related, different tool. `<T extends Number>` on a method or class constrains what `T` may be *and* keeps the type available, so you can use `T` in several positions and the compiler ties them together. The rule of thumb: use a **wildcard** when the type appears once and you do not need to name it; use a **type parameter** when you need to refer to the same type twice.

Code examples

The problem wildcards solve:

JAVA

Copy

```java
// sumBad(List.of(1, 2, 3));      ✗ List<Integer> is not a List<Number>

// ✓ Producer — we only read, so extends.
double sum(List<? extends Number> nums) {
    double total = 0;
    for (Number n : nums) total += n.doubleValue();   // ✓ read as Number
    // nums.add(1);                                    ✗ cannot write
    return total;
}
sum(List.of(1, 2, 3));           // ✓ List<Integer>
sum(List.of(1.5, 2.5));          // ✓ List<Double>

// ✓ Consumer — we only write, so super.
void addNumbers(List<? super Integer> dst) {
    for (int i = 0; i < 5; i++) dst.add(i);           // ✓ write
    // Integer first = dst.get(0);                     ✗ only Object comes
}
addNumbers(new ArrayList<Integer>());   // ✓
addNumbers(new ArrayList<Number>());    // ✓
addNumbers(new ArrayList<Object>());    // ✓
```

Both at once — the canonical signature, and the one to remember:

JAVA Copy

```java
// Copies from a producer into a consumer. This is essentially
// Collections.copy, and it is the clearest example of PECS.
public static <T> void copy(List<? super T> dst, List<? extends T> src) {
    for (T item : src) dst.add(item);
    //      ↑ src PRODUCES T (extends)   dst CONSUMES T (super) ↑
}

List<Integer> ints = List.of(1, 2, 3);
List<Number> nums = new ArrayList<>();
copy(nums, ints);              // ✓ works across the hierarchy
```

Where you have already been using PECS without noticing:

```java
// Stream.map — the mapper PRODUCES R and CONSUMES T:
<R> Stream<R> map(Function<? super T, ? extends R> mapper);

// forEach CONSUMES elements:
void forEach(Consumer<? super T> action);

// Comparator.comparing — the extractor consumes T, produces the sort key:
static <T, U extends Comparable<? super U>> Comparator<T> comparing(
        Function<? super T, ? extends U> keyExtractor);

// The practical payoff of "? super T" on a Comparator:
class Animal { }
class Dog extends Animal { }
Comparator<Animal> byName = …;
List<Dog> dogs = …;
dogs.sort(byName);    // ✓ because sort takes Comparator<? super Dog>
                      //   — a comparator of Animals can order Dogs.
```

Unbounded wildcards and the raw-type trap:

JAVA Copy

```java
// <?> means "some specific unknown type" — you can read as Object and
// call methods that do not involve the type parameter.
void printAll(List<?> list) {
    System.out.println(list.size());       // ✓
    for (Object o : list) System.out.println(o);   // ✓
    // list.add("x");                       ✗ unknown type — no writes but
}

// ✗ A raw List is NOT the same thing. It disables generic checking entire
//   so this compiles and is unsafe:
void broken(List list) { list.add("anything"); }
// Always prefer List<?> when you do not care about the element type.
```

Under the hood

Wildcard capture explains an error message that otherwise looks like nonsense. When the compiler needs to name the unknown type it invents one — `capture#1 of ? extends Number` — and errors mention it. The standard fix is a private *capture helper*: a generic method whose type parameter binds the captured type so it can be used consistently.

JAVA Copy

```java
// ✗ "capture#1 of ?" errors — the compiler cannot prove both ends match.
public static void swap(List<?> list, int i, int j) {
    // list.set(i, list.get(j));
}

// ✓ Capture helper: the type parameter T gives the unknown type a name.
public static void swap(List<?> list, int i, int j) { swapHelper(list, i,

private static <T> void swapHelper(List<T> list, int i, int j) {
    T tmp = list.get(i);
    list.set(i, list.get(j));
    list.set(j, tmp);
}
```

Why `Comparable<? super T>` is everywhere in the JDK is worth understanding, because it looks like noise. If `Employee implements Comparable<Person>` — comparing on a field inherited from `Person` — then `Employee` is not a `Comparable<Employee>`, and a signature demanding that would reject it. `Comparable<? super T>` accepts a comparison defined anywhere up the hierarchy, which is exactly the flexible-and-safe form.

Wildcards never appear in a return type in well-designed APIs. Returning `List<? extends Number>` forces every caller to deal with the wildcard, which spreads the constraint outward for no benefit. Effective Java's guidance is blunt: use wildcards on parameters to make an API flexible, never on returns.

Common mistakes

- **Trying to add to a `? extends` collection.** Only `null` is accepted, and the reason is that the actual element type is unknown.

- Expecting a useful type from a `? super` read. You get `Object`.
- Using a raw type instead of `<?>`. `List<?>` is safe and checked; a raw `List` disables checking.
- **Wildcards in return types.** Pushes the constraint onto every caller.
- **Reaching for a wildcard when a type parameter is needed.** If the same type appears twice, name it with `<T>`.
- **Being surprised by "capture#1" errors.** Use a private generic helper to bind the captured type.
- **Over-wildcarding.** A signature nobody can read is worse than one that is slightly less flexible.

Interview lens

Q: "What is PECS?"

A: Producer Extends, Consumer Super. If a parameter produces values you will read, declare it `? extends T`; if it consumes values you will write, declare it `? super T`. It comes from the compiler knowing only one end of the type range — with `extends` it knows everything coming out is at least a T, so reads are safe; with `super` it knows a T is a legal element for every possible actual type, so writes are safe.

Q: "Why can't you add to a `List<? extends Number>`?"

A: Because the compiler does not know what it really is. It could be a `List<Double>`, in which case adding an `Integer` would corrupt it. Since it cannot prove the write is safe for every type the wildcard could stand for, it forbids all writes except `null`, which is a valid value of every reference type.

Q: "Why are generics invariant when arrays are covariant?"

A: Array covariance was a Java 1.0 decision made before generics existed, and it is unsound — it is only patched at run time by `ArrayStoreException`. Generics are

erased, so there is no run-time check available to patch it. Invariance plus wildcards gives the same flexibility with compile-time safety instead.

Q: "Wildcard or type parameter?"
A: A wildcard when the type appears once in the signature and I never need to name it — it makes the API accept more callers. A type parameter when the same type appears in more than one place and the compiler must tie them together, for example returning the same type it took in.

Q: "What does `List<?>` mean, and how is it different from a raw `List`?"
A: `List<?>` is a list of some specific but unknown type — fully checked, so you can read elements as `Object` and call methods that do not involve the type parameter, but you cannot add anything except `null`. A raw `List` opts out of generic checking altogether, so unsafe adds compile silently. Always prefer `List<?>`.

Q: "Why does `Collections.sort` take `Comparable<? super T>`?"
A: So a comparison defined on a supertype still works. If `Employee implements Comparable<Person>`, then `Employee` is not a `Comparable<Employee>` and a stricter signature would reject it — even though ordering employees by a `Person` field is perfectly sensible. It is PECS applied to the comparison itself.

Cheat-card

- **PECS**: Producer `extends` (read), Consumer `super` (write).
- `? extends T` → read as T, **no writes** (except null).
- `? super T` → write T, **reads give Object**.
- Canonical signature: `copy(List<? super T> dst, List<? extends T> src)`.
- Generics are **invariant** — that is why wildcards exist.
- `List<?>` = unknown but checked. Raw `List` = unchecked. Never raw.

- Wildcard when the type appears **once**; type parameter when it appears **twice**.
- No wildcards in return types.

- "capture#1" error → extract a private generic **capture helper**.

QUICK CHECK — no marks, just for you.

1. Why can you not add an Integer to a List<? extends Number>?

- ◯ Because wildcards make a collection immutable
- ◯ Because Number is abstract
- ◯ Because the actual list might be a List<Double>, and the compiler cannot prove the write is safe for every type the wildcard could stand for
- ◯ Because Integer is not a subtype of Number

Check answer

2. In 'void copy(List<? super T> dst, List<? extends T> src)', why are the wildcards that way round?

- ◯ They are interchangeable; either order compiles
- ◯ extends is always used for the first parameter by convention
- ◯ super allows reading and extends allows writing
- ◯ PECS — src PRODUCES elements so it uses extends; dst CONSUMES them so it uses super

Check answer

3. How does List<?> differ from a raw List?

List<?> is fully checked — you can read as Object but cannot add anything except null — while a raw List disables generic checking entirely, so unsafe adds compile silently

- ○ List<?> is immutable; raw List is mutable
- ○ A raw List cannot be iterated
- ○ They are the same thing with different syntax

Check answer

‹ Previous
Generics and Type Erasure

Next ›
Comparable vs Comparator

LEARN MORE

ARTICLES & DOCS

- Wildcards — The Java Tutorials
  docs.oracle.com
- Wildcard use guidelines — The Java Tutorials
  docs.oracle.com

VIDEOS



PECS Principle For Java Generics - Producer Extends, Consumer Super, Wildcards, Unbounded, Bounded
KnowledgeSharingG



Java Generics Wildcards Explained: PECS & Type Erasure | Java Mastery Series
Beat99.Academy

Hand-picked references for this lesson.